

ReportNow

Project Plan

Version 1.0 approved

Dylan Quek Zhi En

Ang Wei Ming

Chi Seo Hyeon

Choo Kean Yee

Madelyn Cruz Tan

Lim Jun Howe

Tan Yong Huat

School of Computer Science and Engineering,
Nanyang Technological University

12th Oct 2025

Revision History

Revision Number	Date	Primary Author(s)	Comments
1.0	12 th Oct 2025	Dylan Quek Zhi En, Ang Wei Ming, Chi Seo Hyeon, Choo Kean Yee, Madelyn Cruz Tan, Lim Jun Howe, Tan Yong Huat	Initial Project Plan Draft

Table of Contents

1 Introduction	4
1.1 Project Overview	4
1.2 Project Description and Scope	4
2 Project Organisation	5
2.1 Team Structure	5
2.2 Roles and Responsibilities	5
2.3 Team Communication	6
3 Process Definition	7
3.1 Lifecycle Model	7
4 Schedule	8
4.1 Activity Dependencies and Schedule	8
4.2 Work Breakdown Structure	9
4.3 Work Packages	9
4.4 Activity Dependencies	9
4.5 Work Package Details	11
5 Project Estimates	19
5.1 Code Size Estimation using Function Points	19
5.1.1 Unadjusted Function Points	19
5.1.2. Adjusted Function Points	22
5.1.3 Lines of Code	23
5.2 Efforts, Duration and Team Size Estimation	23
5.2.1 Distribution of Effort	23
5.3 Cost Estimates	24
6 Product Checklist	26
7 Best Practices	27
8 Risk Management	28
9 Quality Plan	30
10 Monitoring & Control	31

1 Introduction

1.1 Project Overview

ReportNow is a web and mobile-based platform that enables citizens to submit and track reports, automatically routing them to the relevant government agencies. It bridges current limitations, fosters inclusivity in community participation, and enhances transparency and responsiveness in addressing community issues.

1.2 Project Description and Scope

ReportNow is a web and mobile-based platform that enables citizens to submit and track reports, automatically routing them to the relevant government agencies.

The basis of the all-in-one mobile application platform takes several main features as stated below:

- A robust reporting platform that allows citizens to submit, track, and manage a broad range of public concerns.
- intuitive, user-friendly interface to ensure accessibility and encourage widespread adoption.
- intelligent issue management capabilities, including AI-assisted categorisation, duplicate detection, and crowdsourced feedback, to enhance accuracy and responsiveness.
- Displays the most recent reports on first load.
- User registration will include strong validation measures to ensure reliable participation, rejecting invalid entries such as incorrect email formats or incomplete fields.
- Streamlined reporting workflow so that new users can submit an issue by inputting title, description, location and optional image upload without unnecessary complexity.
- Navigation will be designed for efficiency, allowing users to access detailed report information within three taps or clicks from the home screen.
- AI model trained to categorise issues into predefined types (e.g., potholes, accidents, faulty infrastructure) with a target classification accuracy of at least 80% on labelled data. A duplicate detection mechanism will flag similar or identical issues within seconds of submission, reducing redundancy.
- Users will also be able to upvote reports, creating a real-time crowdsourced ranking that highlights issues of greatest collective concern.

2 Project Organisation

2.1 Team Structure

The following is the list of executive roles, as required by CMM level 3.

- Project Manager: Madelyn Cruz Tan
- Lead Developer: Ang Wei Ming
- Front-end Developer: Dylan Quek Zhi En
- Back-end Developer: Chi Seo Hyeon
- QA Manager: Tan Yong Huat
- QA Engineer: Choo Kean Yee
- Release Engineer/Manager: Lim Jun Howe

2.2 Roles and Responsibilities

Project Manager: Madelyn Cruz Tan

- Oversees project progress and delivery of product

Lead Developer: Ang Wei Ming

- Overall technical lead, responsible for technical aspects of product release

Front-end Developer: Dylan Quek Zhi En

- Mobile app or web app front-end programming
- Participate in the entire SDLC, generating work products including documentation, source code, unit and integration tests

Back-end Developer: Chi Seo Hyeon

- Server, application, and database programming
- Participate in the entire SDLC, generating work products including documentation, source code, unit and integration tests

QA Manager: Tan Yong Huat

- Overall product and process quality
- Implementation of QA processes

QA Engineer: Choo Kean Yee

- Devise test plans
- Conduct tests

Release Engineer/Manager: *Lim Jun Howe*

- Create baselines and build and integrate changes for delivery
- Manage releases of product (or prototype in our case)

2.3 Team Communication

Communication is critical to the development of the project. ReportNow's main channels of communication include:

- Meetings to be held at least once every 2 weeks and as agreed upon by the team
- Source codes and issues are tracked with Trello and GitHub.
- Group announcements and updates are sent through Telegram.
- Split up into subgroups as necessary, to work more cooperatively on specific problems.

3 Process Definition

3.1 Lifecycle Model

ReportNow adopts an Agile, iteration-driven lifecycle in which the product evolves through small, incremental releases. Work is decomposed into user stories that are delivered in short sprints, allowing us to validate assumptions early, surface defects sooner, and continuously refine scope with stakeholder input. Testing, security checks, and accessibility reviews begin as soon as a story is in development and continue through integration and user acceptance on a staging environment. Each sprint begins with planning to select a coherent, outcome-oriented goal and ends with a review and a retrospective to inspect the increment and adapt both the product backlog and our working methods.

Agile is particularly appropriate for ReportNow because timelines are tight and several dependencies, such as map, media, and authentication providers, may change during development. The architecture is intentionally modular so that independent feature tracks (user login, report, review, etc.) can progress together. In practice, all development occurs on short-lived feature branches created from a protected main; developers push commits to their own branches and open pull requests for review. Continuous integration enforces type safety, code quality, automated tests, and security scans on each pull request; only after passing these gates and peer review are changes merged into main. Successful merges auto-deploy to staging for end-to-end and accessibility checks, and then release to production behind flags. This cadence provides both flexibility and traceability, making it easier to incorporate feedback, manage change, and maintain a reliable delivery rhythm.

4 Schedule

4.1 Activity Dependencies and Schedule

The following project scheduling timeline has been delivered in a Gantt Chart below:

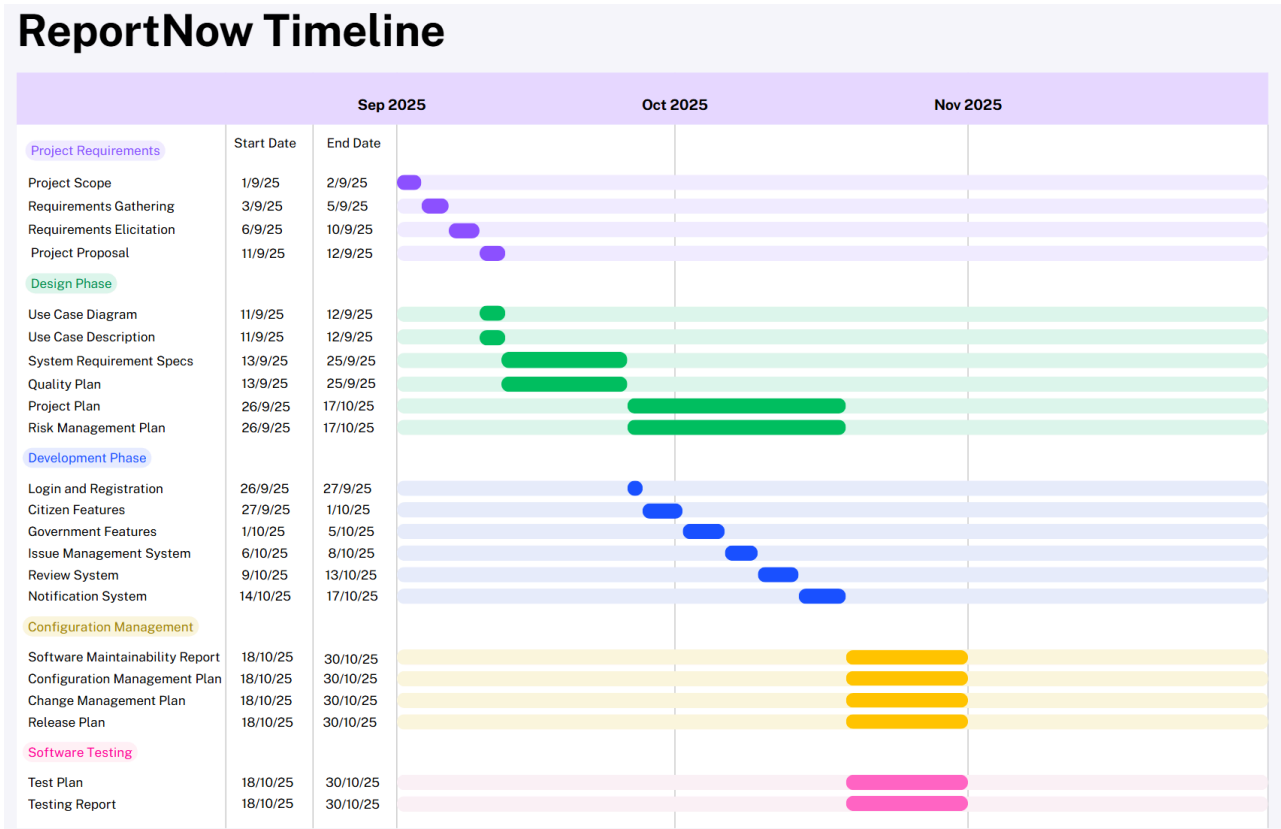


Figure 1. Gantt Chart for Project Deliverables

4.2 Work Breakdown Structure

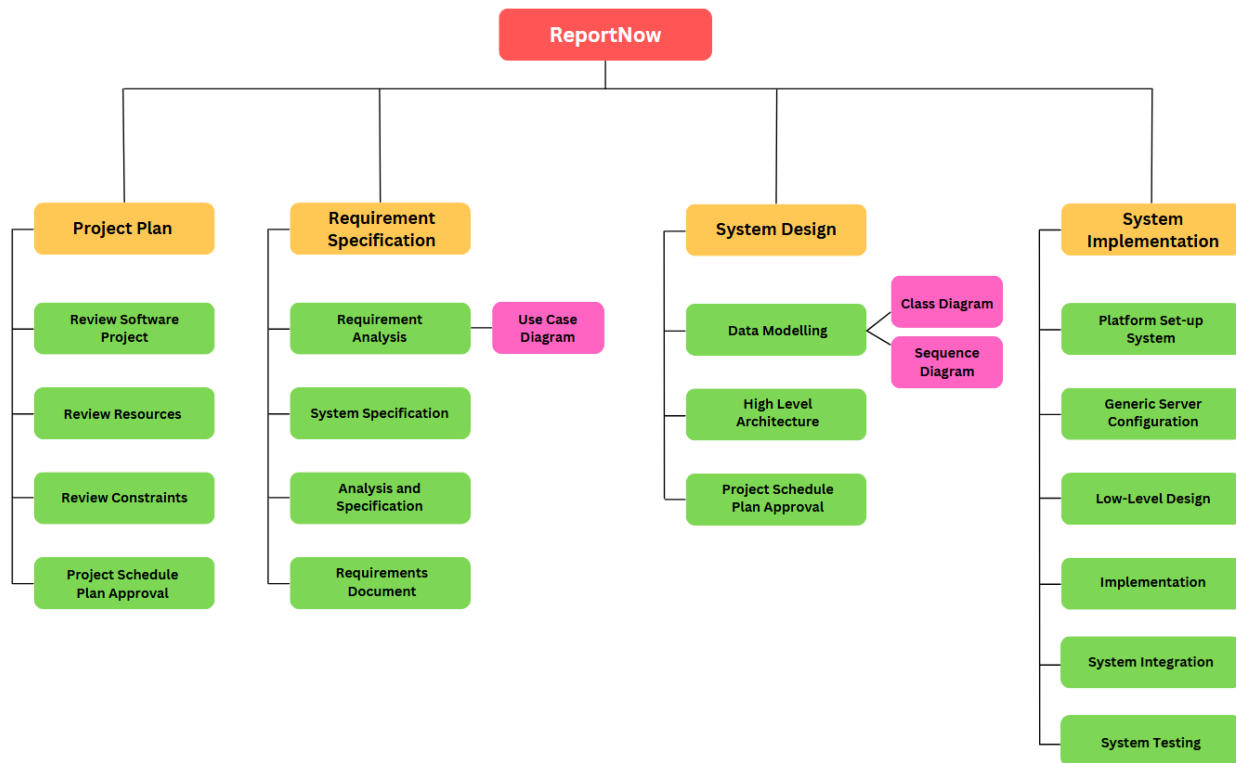


Figure 2. Work Breakdown Structure

4.3 Work Packages

The entire project work is broken down into the important phases of the software development life cycle. They include the following:

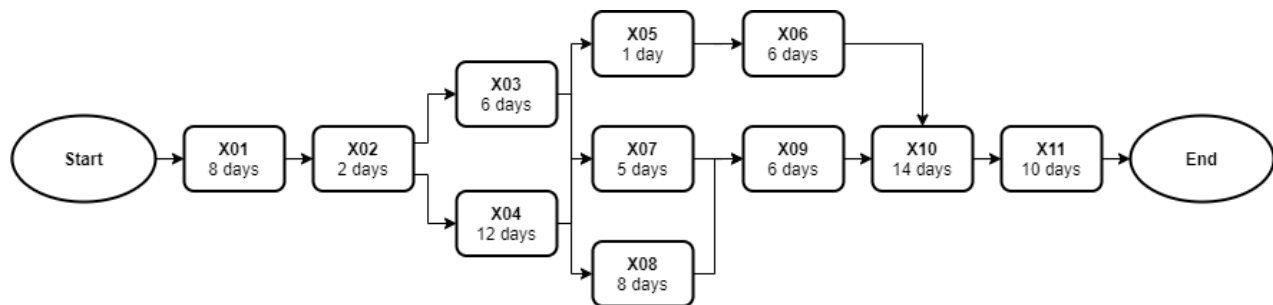
1. Project Proposal
2. Requirement Elicitation
3. Software Quality Plan
4. System Requirement Specification
5. Risk Management
6. Project Plan
7. User Interface Design
8. Technical Architecture
9. Data Modeling
10. Coding & Unit Testing
11. Integration & System Testing

4.4 Activity Dependencies

The following table describes the dependencies of the deliverable work packages:

Work Package #	Work Package Description	Duration	Dependencies
X01	Project Proposal	8 days	-
X02	Requirements Elicitation	2 days	X01
X03	Software Quality Plan	6 days	X02
X04	System Requirement Specification	12 days	X02
X05	Risk Management	1 day	X03, X04
X06	Project Plan	6 days	X03, X04, X05
X07	User Interface Design	5 days	X03, X04
X08	Technical Architecture	8 days	X03, X04
X09	Data Modelling	6 days	X08
X10	Coding & Unit Testing	14 days	X06, X07, X09
X11	Integration & System Testing	10 days	X10

The following Activity Network Diagram describes the above in more graphical detail:



Critical Path Analysis:

Work Package	Earliest Start	Earliest Finish	Latest Start	Latest Finish	Slack Time
X01	Start of day 0	End of day 7	Start of day 0	End of day 7	0 days
X02	Start of day 8	End of day 9	Start of day 8	End of day 9	0 days
X03	Start of day 10	End of day 15	Start of day 23	End of day 28	13 days
X04	Start of day 10	End of day 21	Start of day 10	End of day 21	0 days
X05	Start of day 22	End of day 22	Start of day 29	End of day 29	7 days
X06	Start of day 23	End of day 28	Start of day 30	End of day 35	7 days
X07	Start of day 22	End of day 26	Start of day 25	End of day 29	3 days
X08	Start of day 22	End of day 29	Start of day 22	End of day 29	0 days
X09	Start of day 30	End of day 35	Start of day 30	End of day 35	0 days
X10	Start of day 36	End of day 49	Start of day 36	End of day 49	0 days
X11	Start of day 50	End of day 59	Start of day 50	End of day 59	0 days

Critical Path: Start → X01 → X02 → X04 → X08 → X09 → X10 → X11 → End

4.5 Work Package Details

Work packages are listed below. A team member, indicated in bold, has been assigned as the primary responsible for each work package and will coordinate that package.

Project	ReportNow
Work Package	X01 - Project Proposal
Assigned To	All members
Effort	8 Person Days
Start Date	29-08-2025
Purpose	To define project objectives, identify project scope, and determine target specifications.
Inputs	Problem identified, some possible solutions and requirements from users.
Activities	1. Draft problem statement. 2. Development of a Business case. 3. Draft design objectives, based on the selected solution. 4. Define the Technical approaches of the solutions. 5. Establish brief project management.
Outputs	Project Proposal

Project	ReportNow
Work Package	X02 – Requirements Elicitation
Assigned To	Madelyn, Wei Ming
Effort	2 Person Days
Start Date	06-09-2025
Purpose	To gather requirements that can help further refine the details of the

	project, various requirement elicitation techniques will be used
Inputs	User requirements, all outputs from work package Project Proposal (X01)
Activities	1. Conducting research on existing solutions to understand the requirements of the software. 2. Brainstorming on the possible requirements that might be useful for the user. 3. Building a prototype to gather more requirements and build a common understanding of the requirements of user.
Outputs	A list of requirements gathered from research, brainstorming, prototyping and observations

Project	ReportNow
Work Package	X03 - Software Quality Plan
Assigned To	Jun Howe, Yong Huat, Kean Yee
Effort	6 Person Days
Start Date	08-09-2025
Purpose	To establish the goals, processes, and responsibilities required to implement effective quality assurance functions for the ReportNow project. To ensure a project complies with the standard and is of a certain quality.
Inputs	All outputs from work package Requirement Elicitation(X02)
Activities	1. Establish project purpose and scope. 2. Identify the management organisational structure, its roles and responsibilities, and the software quality tasks to be performed. 3. Define documentation standards that the project shall follow. 4. Establish tools and methodologies on how software quality can be upkeep with.
Outputs	Software Quality Plan

Project	ReportNow
Work Package	X04 - System Requirement Specifications
Assigned To	Madelyn, Wei Ming, Dylan, Seo Hyeon
Effort	12 Person Days
Start Date	08-09-2025
Purpose	To establish a common understanding between the customer and the software project team of the customer's requirements to be addressed by the project. Information gathered from requirement elicitation will be formally documented.
Inputs	All outputs from work package Requirement Elicitation (X02)
Activities	1. Document problem statement, background, and overall description 2. Identify the constraints of the project 3. With elicited requirements, draft operational, functional, input, process, output, hardware, software, and deployment requirements.
Outputs	A written document of the System Requirements Specification

Project	ReportNow
Work Package	X05 - Risk Management Plan
Assigned To	Wei Ming, Dylan, Kean Yee, Seo Hyeon
Effort	1 Person Days
Start Date	20-09-2025
Purpose	To identify and analyse all risks (positive or negative) associated with the project. The plan will outline how these risks would be mitigated and how risk management activities will be performed, recorded, and monitored

Inputs	All outputs from work package Software Quality Plan (X03), System Requirements Specification Details (X04)
Activities	1. Identify and analyse the risks associated with the cost, resource scheduling, and technical aspects. 2. Determine methodologies to mitigate the risks identified.
Outputs	A written document of the Risk Management Plan

Project	ReportNow
Work Package	X06- Project Plan
Assigned To	Madelyn, Jun Howe, Yong Huat
Effort	6 Person Days
Start Date	21-09-2025
Purpose	To define the overview of the project process in the document and give a project schedule.
Inputs	All outputs from work package Software Quality Plan (X03), System Requirements Specification Details (X04) and Risk Management(X05)
Activities	This work package includes: 1. Define project overview and scope 2. Identify the project team organisation 3. Model after project processes 4. Plan project schedule 5. Estimate project costs 6. Identify and allocate available resources by estimating effort, duration, and team size 7. Define product checklist 8. Identify best practices 9. Brief introduction to risk management 10. Reiterate the quality plan 11. Define monitoring and control methodologies
Outputs	A written document of the Project Plan

Project	ReportNow
Work Package	X07 - User Interface Design
Assigned To	Dylan, Madelyn
Effort	5 Person Days
Start Date	20-09-2025
Purpose	To create an effective communication medium between the system and the customer. The initial UI prototype will serve as a reference point for how the system product will look and how customers interact with the product. This will help to mitigate confusion or disagreement.
Inputs	All outputs from work package Software Quality Plan (X03), System Requirements Specification Details (X04)
Activities	1. Referencing the Customer Requirements and System Requirements Specification Details, draft an initial Hi-Fidelity prototype design. 2. Seek opinions from different stakeholders and further refine the user interface design with design principles and Shneiderman's Eight Golden Rules 3. Repeat Steps 1 and 2 until the user interface design is coherent with the requirements. 4. Build the user interface (screen layouts only) as a minimum viable product.
Outputs	Working Prototype of User Interface (front end only)

Project	ReportNow
Work Package	X08 - Technical Architecture
Assigned To	All Members
Effort	8 Person Days
Start Date	20-09-2025

Purpose	To perform high-level architectural design, one must develop an overview of how each subcomponent in the system interacts with each other.
Inputs	All outputs from work package Software Quality Plan (X03), System Requirements Specification Details (X04)
Activities	1. High-level design entails defining the architecture of the software system and 2. Defining the various components required for the system and how they are interrelated to each other. 3. Deciding on the software and hardware infrastructures, such as what operating system on which the software is built, the language used to implement the software, and so on. Designers must consider topics including maintainability, portability, and reusability will be addressed here as well.
Outputs	High-level design and Architectural specification

Project	ReportNow
Work Package	X09 - Data Modelling
Assigned To	All Members
Effort	6 Person Days
Start Date	28-09-2025
Purpose	To develop the data model which forms the basis for the project database.
Inputs	All outputs from work package Technical Architecture (X08)
Activities	1. Perform analysis on data flow and entity relationships. 2. Consider data types and tables that are required to represent stored data in the system.
Outputs	A written document for data modelling

Project	ReportNow
Work Package	X10 - Coding & Unit Testing
Assigned To	All Members
Effort	14 Person Days
Start Date	04-10-2025
Purpose	To implement the system as per the Requirement Specifications, system Architecture and UI design. This work package includes such additional activities as preliminary unit testing to test out the various quality assurance testing factors, such as optimality and reliability for individual components, before integration.
Inputs	All outputs from work package Project Plan(X06), User Interface Design (X07), Data Modelling (X09)
Activities	1. Programmers will implement the modules according to the design specifications noted in the System Requirement Specifications. 2. Programmers shall carry out unit test plans.
Outputs	Source code, header files, integrated system test plan, and test cases

Project	ReportNow
Work Package	X11 - Integration & System Testing
Assigned To	Kean Yee, Jun Howe
Effort	10 Person Days
Start Date	18-10-2025
Purpose	To integrate the various components into the system and build the data flow between them. To identify and fix logical and syntactical errors produced during the implementation of the System and setting up drivers

	and stubs to see how the module responds to various inputs. Black box testing, as well as white box testing, might be conducted to check for logical errors. All the testing procedures will be documented in the Test Plan report. If problems are found, they will be noted and fixed at the earliest possible time.
Inputs	All outputs from work package Coding & Unit Testing (X10).
Activities	1. The Integration testing team may try to simulate how a user might interact with the system. 2. Development of stubs and drivers as well, but here this is more geared towards the higher (overall system) level. 3. Testers will also examine issues such as system performance and integrity. Heuristics assessment plays an important role in this work package, as intelligence components will define eventual system success.
Outputs	A written document of the Requirement Test Coverage Report.

5 Project Estimates

5.1 Code Size Estimation using Function Points

We calculated the unadjusted function point based on the complexity of the functions provided by this system. Code size is then estimated by the adjusted function point.

5.1.1 Unadjusted Function Points

ReportNow supports the following proposed user functions:

- Submit an incident report (with description, image, and location)
- Track report status and updates
- Register and log in as a user
- View all reports in a map/list format
- Receive notifications on status changes
- AI-based categorisation and duplicate detection
- Vote and comment on community reports
- Agency users manage and resolve reports

The measure of unadjusted function points is based on five primary component elements of these functions: Inputs, Outputs, Inquiries, Logical Files, and Interfaces. Each element ranges from Low Complexity, Medium Complexity, to High Complexity. The detailed evaluation of the complexity is as follows:

Rating Inputs:

- Submitting new reports
- Logging in / registering an account
- Voting or commenting on reports
- Updating report details
- Agency-side status updates
- Account registration

Files Type Referenced (FTR)	Data Elements		
	1 - 4	5- 15	Greater than 15
Less than 2	Low (3)	Low (3)	Average (4)
2	Low (3)	Average (4)	High (6)
Greater than 2	Average (4)	High (6)	High (6)

Rating Outputs:

- Report feed and map visualisation
- Status updates (“In Progress”, “Resolved”)
- Analytics charts for administrators
- AI categorisation labels and duplicate warnings
- Push/email notifications

File Types Referenced (FTR)	Data Elements		
	1-5	6-19	Greater than 19
Less than 2	Low (4)	Low (4)	Average (5)
2 or 3	Low (4)	Average (5)	High (7)
Greater than 3	Average (5)	High (7)	High (7)

Rating Inquiries:

- Search reports by keyword or category
- Filter by status or location
- Forgot password

File Types Referenced (FTR)	Data Elements		
	1-5	6-19	Greater than 19
less than 2	Low (3)	Low (3)	Average (4)
2 or 3	Low (3)	Average (4)	High (6)
Greater than 3	Average (4)	High (6)	High (6)

Rating Logical Files:

- User Accounts
- Reports Table
- Votes and Comments Table
- Agency Responses and Status Logs

Record Element Types (RET)	Data Elements		
	1 to 19	20 - 50	51 or More
1 RET	Low (7)	Low (7)	Average (10)
2 to 5 RET	Low (7)	Average (10)	High (15)
6 or More RET	Average (10)	High (15)	High (15)

Rating Interfaces:

- Google Maps API (for geolocation)
- Cloud
- LLM Categorization

Record Element Types (RET)	Data Elements		
	1 to 19	20 - 50	51 or More
1 RET	Low (7)	Low (7)	Average (10)
2 to 5 RET	Low (7)	Average (10)	High (15)
6 or More RET	Average (10)	High (15)	High (15)

Summary of the above analysis:

Element	Complexity	Detail
Inputs	Average	Submitting new reports
	Low	Logging in / registering an account
	Average	Voting or commenting on reports
	Low	Updating report details
	Low	Agency-side status updates
	Low	Account registration
Logical Files	Average	User Accounts
	Average	Reports Table
	Average	Voters and Comments Table
	Average	Agency Responses and Status Logs
Outputs	Average	Report feed and map visualisation
	Low	Status updates (“In Progress”, “Resolved”)
	Average	Analytics charts for administrators
	Average	AI categorisation labels and duplicate warnings
	Low	Push / email notifications
Inquiries	Low	Search reports by keyword or category
	Low	Filter by status or location
	Low	Forgot Password
Interfaces	Low	Google Maps API (geolocation)
	Average	Cloud
	Average	LLM Categorisation

Calculation of Unadjusted Function Points:

Characteristic	Low		Medium		High	
Inputs	4	× 3	2	× 4	0	× 6
Outputs	2	× 4	3	× 5	0	× 7
Inquiries	3	× 3	0	× 4	0	× 6
Logical Files	0	× 7	4	× 10	0	× 15
Interfaces	1	× 5	2	× 7	0	× 10
Unadjusted FP	34		77		0	
Total=L+M+H	111					

5.1.2. Adjusted Function Points

Influence Factors	Score	Detailflutter
Data Communications	5	The application uses internet connectivity between the client and the backend.
Distributed Functions	4	ReportNow supports distributed processing across front-end, API server and cloud database.
Performance	4	Response time is critical for map rendering and API updates, requiring optimisation.
Heavily used	3	Multiple concurrent submissions and reads are expected during peak hours.
Transaction rate	2	Average workload, with brief spikes when reports surge.
Online data entry	5	Over 70 % of transactions are interactive report submissions.
End-user efficiency	4	UI design follows usability heuristics and accessibility best practices.
Online data update	4	Users and agencies update and resolve reports in real-time.
Complex processing	3	AI categorisation and duplicate-detection add complexity.
Reusability	4	Components (login, map, report cards) are reusable modules across platforms.
Installation Ease	3	No special considerations were stated by the user, but the backend configuration requires setup.
Operational Ease	4	Automatic backups, cloud logging, and minimal manual intervention.
Multiple sites	2	Deployed for multiple agencies; minimal geographic configuration.
Total score	47	
Influence Multiplier $= \text{Total score} \times 0.01 + 0.65 = 47 \times 0.01 + 0.65 = 1.12$		
Adjusted FP $= \text{Unadjusted FP} \times \text{Influence Multiplier} = 111 \times 1.12 = 124.32$		

Scoring (0 – 5)
0 = No influence
1 = Insignificant influence
2 = Moderate influence
3 = Average influence
4 = Significant influence
5 = Strong influence

5.1.3 Lines of Code

Lines of Code = $124.32 \text{ FP} \times 53 \text{ LOC/FP} = 6589 \text{ LOC (approx)}$.

5.2 Efforts, Duration and Team Size Estimation

To estimate the effort and duration required for the project, we use function points as the basis to calculate Effort, Duration, Team Size and finally the schedule. The estimates are expanded to account for project management and extra contingency time to obtain the total average effort estimates. From these averages, the duration of each work package in working days is estimated based on the following calculations.

- All members are expected to work 7 days a week.
- Effort = Size / Production Rate = $(6589 \text{ LOC}) / (31 \text{ LOC/PD})^1 = 212.55 \text{ PD}$
- Duration = $3 \times (\text{Effort})^{1/3} = 3 \times (212.55)^{1/3} = 18 \text{ Days}$
- Initial schedule = $18 \text{ Days} / 7 \text{ days a week} = 2.57 \text{ Weeks}$
- Team size = $212.55 \text{ PD} / 18 \text{ D} = 12 \text{ Persons}$
- Working hours include 4 hours in a working day.
- Total person-hours (PH) = $212.55 \text{ PD} \times 4 \text{ hours} = 850.2 \text{ PH}$

5.2.1 Distribution of Effort

1990s Industry Data	Work Package	Distribution	Estimates
Preliminary Design 18%	Project Proposal	4%	19.85
	Requirement Elicitation	1%	4.96
	Software Quality Plan	3%	14.89
	System Requirement Specification	6%	29.78
	Risk Management	1%	4.96
	Project Plan	3%	14.89
Detailed Design 25%	User Interface Design	7%	34.74
	Technical Architecture	11%	54.59
	Data Modelling	7%	34.74
Code & Unit Testing 26%	Coding & Unit Testing	26%	129.03
Integration & Test 31%	Integration & System Testing	31%	153.84
	Extrapolated Total Effort:		496.26
	2% for project management		9.93
	3% for contingency		14.89
	Total Effort:		521.08

These duration estimates assume that each team member works an equal amount on any given work package.

¹ Lines of code per Person Day statistics based on Industrial Benchmarks, 1997: 31 LOC/PD for United States; 62 LOC/PD for Canada

5.3 Cost Estimates

Hardware:

Developer workstations:

7 - Dell Precision Workstation 330	Total \$0.00
Pentium IV 1.4GHz single processor	
256 MB RAM	
20.8GB IDE drive	

Software:

Development Tools:

Prisma	\$0.00
NextJS	\$0.00

Documentation/version control tools:

GitHub	\$0.00
Tortoise SVN	\$0.00
Diagrams.net	\$0.00
Visual Paradigm	\$0.00
Figma	\$0.00
Android Studio	\$0.00
Trello	\$0.00
Team Gantt	\$0.00

Software License Provided by Third Party:

Microsoft Office	\$0.00
------------------	--------

Other Resources:

Staff:

9 Employees with 1164.45 working hours with \$18.00/hour	\$20,960.10
--	-------------

Stationery:

Paper, photocopying and other miscellaneous costs	\$50.00
---	---------

Total: \$21,010.10

The mobile application will be run on the user's mobile device. ReportNow is not responsible in any way for supplying said device, nor is it responsible for any issues that arise due to the fault of the user's device. ReportNow's responsibility is only to develop the mobile application to satisfy all stated functional and system requirements. Hence, any costs incurred due to faults of the user's device are to be borne by the user.

6 Product Checklist

The plan is that the items listed below will be delivered by the stated deadlines.

<u>Lab</u>	<u>Project Deliverable</u>	<u>Estimated Deadline</u>
All lab submissions	Meeting Minutes	Every meeting
	Trello Backlog	Updated after every lab
Lab 1	Team Registration	29 th August 2025 (during Lab 1)
	Team Information	11 th September 2025 (before Lab 2)
	Project Proposal	
	Use Case Model & Description	
Lab 2	System Requirement Specification	25 th September 2025 (before Lab 3)
	Quality Plan	
Lab 3	Project Plan	16 th October 2025 (before Lab 4)
	Risk Management Plan	
	Risk Management Log	
	Simple Prototype Demo	
Lab 4	Design Report on Software Maintainability	30 th October 2025 (before Lab 5)
	Configuration Management Plan	
	Change Management Plan	
	Release Plan	
	Prototype Demonstration	
Lab 5	Presentation Slides	30 th October 2025 (before Lab 5)
	Test Plan	
	Test Cases and Requirements Test Coverage Report	
	Peer Review	

7 Best Practices

Throughout the planning and development phases of ReportNow, the team has developed certain criteria of expectations, or Best Practices, to ensure the quality and successful delivery of the project and relevant documents.

Best Practices
Documentation of the planning and development phases; all documentation should be in a standardised format.
Project requirements will be carefully assessed, with thorough inspections for ambiguity, consistency, accuracy, and project completeness. The requirements documentation must include complete functional specifications.
Documentation of update history - All changes to documents, such as designs, test records and source code, must be recorded with proper details so that they are trackable. - Updates to documents and code must be approved before being applied to check for compliance with specifications and ensure quality. - Usage of the configuration management system and make processes. This facilitates efficient and convenient maintenance.
Refrain from making under/overestimates. This applies to various aspects of the planning and development phases, including but not limited to: deadlines, estimates on required effort and work division.
Ensure and maintain simplicity where possible, such as minimising unnecessary interface complications that might lead to delays in user interactions and hindrance of user experience. This applies to the functionality of the product; unnecessary features (or design) that overdo planned requirements might lead to undesired complexity.
Utilisation of coding principles, such as compliance to coding standards (naming conventions, formatting, comments), usage of version control, and code documentation.
Code modularity: During the development of the product, the program must be modularised for the convenience of maintenance and future updates. This also facilitates efficient debugging and integration processes during the deployment of the product.
Software testing will use both black and white box testing methods. It will involve unit, functional, integration and acceptance testing.
Code reviews are necessary in the process of software development. Modularisation, as mentioned earlier, would enable efficient and convenient code reviews, whilst simplifying the process of division of workload amongst team members.
Rollback plans are necessary to ensure that the product can recover from failed deployments.

8 Risk Management

Besides the general risk management, the following risks have been identified for the ReportNow project:

Unable to meet the Project Deadline

Probability: Medium

Impacts: High

Risk Reduction: Conduct regular progress reviews and set internal milestones to detect slippage early.

Hardware failure, e.g., Hard Drive corrupted

Probability: Low

Impacts: High

Risk Reduction: Maintain version control and off-site backups to restore work quickly.

Feature creep

Probability: Medium

Impacts: High

Risk Reduction: Implement change control and require sponsor approval for new features.

Updates to the code were not communicated to every member

Probability: High

Impacts: Medium

Risk Reduction: Use shared repositories and weekly sync meetings to align updates.

Staff Unavailability

Probability: Medium

Impacts: Medium

Risk Reduction: Redistribute workload or bring in temporary support when necessary.

System not meeting requirements

Probability: Low

Impacts: High

Risk Reduction: Conduct user acceptance testing (UAT) early and review requirements frequently.

Poor code design (e.g., high coupling)

Probability: Low

Impacts: Medium

Risk Reduction: Adopt a modular architecture and enforce peer reviews.

Miscommunication of requirements from stakeholders

Probability: Low

Impacts: High

Risk Reduction: Hold validation sessions with stakeholders before finalising features.

Sensitive data being leaked or hacked

Probability: Low

Impacts: High

Risk Reduction: Implement encryption, secure APIs, and periodic vulnerability scans.

Incomplete Documentation

Probability: Low

Impacts: Low

Risk Reduction: Maintain living documentation and assign review responsibility per sprint.

Inconsistent Programming Style

Probability: Low

Impacts: Medium

Risk Reduction: Adopt a team coding standard and automated linting tools.

Changes in government data policy or API access

Probability: Medium

Impacts: High

Risk Reduction: Monitor policy updates and design flexible API abstraction layers.

User impersonation or spam reports

Probability: Low

Impacts: High

Risk Reduction: Use 2FA and CAPTCHA during user registration and reporting.

AI misclassifies or fails to detect duplicate reports

Probability: Medium

Impacts: Medium

Risk Reduction: Periodically retrain the AI model and allow manual overrides.

Non-compliance with PDPA or Smart Nation security guidelines

Probability: Low

Impacts: High

Risk Reduction: Consult the PDPA compliance checklist and implement proper data handling procedures.

Inaccurate cost or schedule estimation

Probability: Medium

Impacts: Medium

Risk Reduction: Use progressive estimation and contingency buffers.

Inadequate test coverage or late defect discovery

Probability: Medium

Impacts: High

Risk Reduction: Integrate unit and system testing early in the development cycle.

9 Quality Plan

The team will achieve quality assurance by conducting tests on the requirements set for the product. The specific procedures and details will be provided in the Quality Plan.

Specific test procedures and details shall be provided in the Module/System Test Plan.

In addition, the team will make use of the following testing methodologies:

- **Unit Testing** - testing of individual system components for their functionality.
- **Integration Testing** - testing of compatibility and interactions between individual components.
- **System Testing** - testing the whole system as a single integrated program.
- **Validation Testing** - testing to ensure individual components meet the user's requirements and have appropriate responses to various scenarios.
- **User Acceptance Testing** - testing of the prototype by users.

These tests will be conducted under realistic test cases that the users might input or experience while using the product. The test data will be documented and will play a crucial role in the final stages of the development phase.

10 Monitoring & Control

Effective monitoring and control ensure that ReportNow progresses according to plan and that emerging issues are promptly addressed. The following key practices will be used throughout the project lifecycle:

Quantitative Measurement of Resource Consumption

The time taken to develop and test features will be tracked against milestone progress. Comparing estimated and actual time consumption will help assess project efficiency and identify early signs of delay.

Identification of Major Project Risks

Risks are documented and continuously updated in the Risk Management Log. Early recognition of high-priority risks allows the team to implement mitigation or contingency measures before they escalate.

Regular Progress Reviews

The ReportNow team will conduct weekly meetings to review ongoing work regarding planning, development, testing, and documentation. These sessions will track milestone completion, resource utilisation and potential setbacks requiring more attention.

Timeline Planning and Task Decomposition

Project activities are decomposed into smaller, measurable tasks within the Work Breakdown Structure (WBS). This hierarchical breakdown enables more accurate effort estimation and balanced task distribution. During implementation, progress on these subcomponents will be used as an indicator of overall project health.